

Amazon Fresh Analytics

Project Title: Amazon User Data Analytics

Name: Hariharan S

Domain: E-commerce and Retail Analytics

Language: SQL

Tools: My SQL Workbench

Problem Statement:

Amazon Fresh is an e-commerce platform specializing in groceries and daily essentials. As the platform expands, its management seeks to optimize operations, enhance customer satisfaction, and boost revenue. The challenge lies in effectively managing large volumes of data, including customer profiles, product inventories, supplier information, orders, and customer reviews. The problem is to design a relational database for Amazon Fresh that organizes this data efficiently, allows for meaningful analysis, and supports critical business decisions. This project aims to solve key business problems, such as identifying top-performing products, analyzing customer purchasing patterns, and ensuring inventory management aligns with demand trends.

Business Use Cases:

- **Customer Insights:** Analyze purchasing patterns to identify top customers and cities for targeted marketing.
- **Product Management:** Monitor product demand, stock levels, and performance of product categories.
- **Supplier Analysis:** Identify key suppliers and optimize supply chain operations.
- **Revenue Analysis:** Generate revenue trends and identify high-performing regions or categories.
- **Customer Satisfaction:** Use review data to enhance product offerings and improve customer experience.

Data Set:

1. **Customers.csv:** Customer details including demographics and Prime membership.
2. **Products.csv:** Product information such as categories and stock levels.
3. **Suppliers.csv:** Supplier details and locations.

4. **Orders.csv:** Order details including customer and date.
5. **Order_Details.csv:** Product-specific order information.
6. **Reviews.csv:** Customer ratings and reviews for products

Data Set Explanation:

1. **Customers Dataset**

Information about customers, like their names, cities, and Prime membership status.

2. **Products Dataset**

Information about products available, including price and stock.

3. **Suppliers Dataset**

Information about suppliers, like their names and contact details.

4. **Orders Dataset**

High-level details of orders placed by customers.

5. **Order Details Dataset**

Specific products within each order.

6. **Reviews Dataset**

Feedback from customers about products.

Tools Used:

MySQL Workbench:

Create databases, tables, columns, and relationships visually.

Approach:

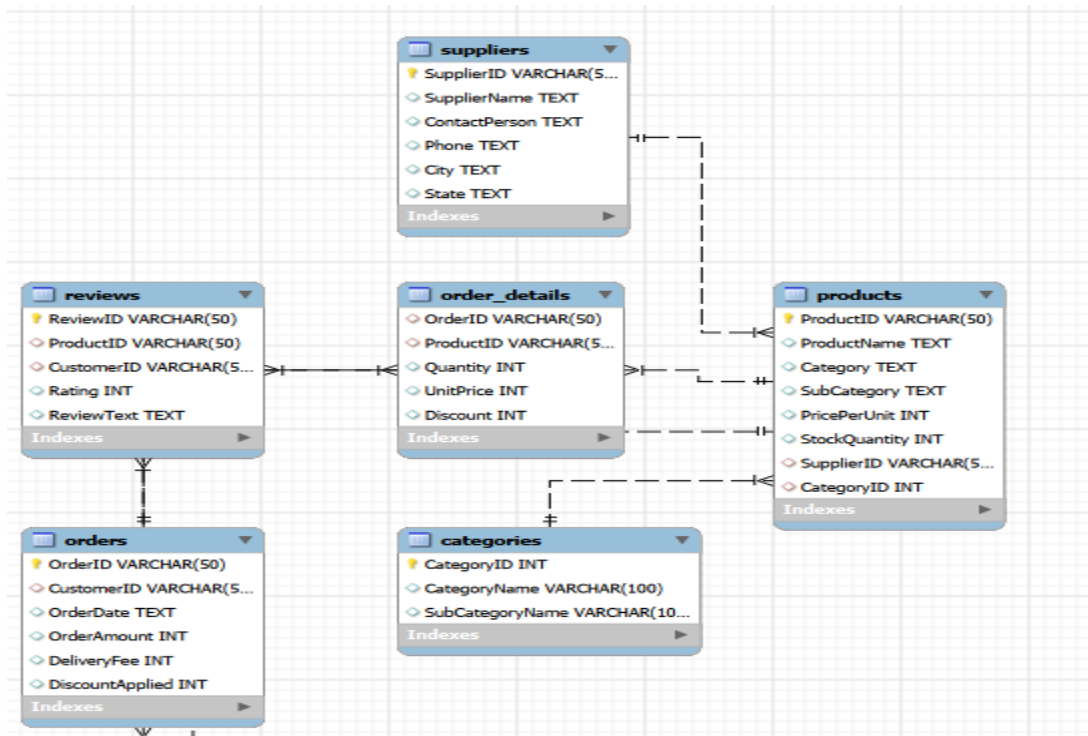
1.Data Modeling and Basic Queries

1.1 ER Diagram for Amazon Fresh Database

The ER (Entity Relationship) Diagram represents the relationships between tables in the Amazon Fresh database system.

Main Entities and Relationships

- **Customers** place **Orders**
- **Orders** contain **Order Details**
- **Products** are included in **Order Details**
- **Suppliers** supply **Products**
- **Customers** give **Reviews** for **Products**



1.2 Primary Keys and Foreign Keys

1.2.1 Primary Keys

Table Name	Primary Key
Customers	CustomerID
Products	ProductID
Suppliers	SupplierID
Orders	OrderID
Order_Details	OrderDetailID
Reviews	ReviewID

1.2.2 Foreign Key

Table Name	Foreign Key	References
Orders	CustomerID	Customers(CustomerID)
Order_Details	OrderID	Orders(OrderID)
Order_Details	ProductID	Products(ProductID)
Products	SupplierID	Suppliers(SupplierID)
Reviews	CustomerID	Customers(CustomerID)
Reviews	ProductID	Products(ProductID)

1.3. SQL query to get Specific Data:

➤ Retrieve All Customers from a Specific City

Select * from customers **where** City = "West Daniel";

➤ Fetch All Products Under the “Fruits” Category

Select ProductName, Category ,Priceperunit, StockQuantity from products
where Category = "Fruits";

2.Data Definition Language (DDL) and Constraints

- **CustomerID as the primary key**

Alter table Customers

Add constraint New_constraint **primary key** (CustomerID);

- **Ensure Age cannot be null and must be greater than 18.**

Alter table customers

modify column age int not null;

Select * From customers

Where Age <= 18;

3.Data Manipulation Language (DML)

- **Insert 3 new rows into the Products table using INSERT statement**

SELECT * FROM suppliers;

INSERT INTO suppliers (SupplierID)

VALUES

('jhvcjq762-hjabjc-23rfcqa32-sedvaqw5'),

('afbh12-324bjkwsj-32rfa34r-3rf24trf'),

('vkijbqw34-23rfcwww-q3rdf21-f1qert5');

- **Update the stock quantity of a product where ProductID matches a specific ID.**

Update products

set StockQuantity = '75' where ProductID = '1003';

- **Delete a supplier from the Suppliers table where their city matches a specific value.**

select City from suppliers;

delete from suppliers where City = "Jeffreytown";

4. SQL Constraints and Operators

- **Add a CHECK constraint to ensure that ratings in the Reviews table are between 1 and 5.**

alter table reviews

add constraint CHK_rating **check** (Rating between 1 and 5);

- **Add a DEFAULT constraint for the PrimeMember column in the Customers table (default value: "No")**

```
alter table customers
modify PrimeMember varchar(10) default 'No';
```

5. Clauses and Aggregations

- **WHERE clause to find orders placed after 2024-01-01**
select * from orders where OrderDate = 2024-01-01;
- **HAVING clause to list products with average ratings greater than 4.**
select p.ProductID , p.ProductName , avg(rating) as AVG_Rating from reviews as
r join products as p ON
r.ProductID = p.ProductID group by p.ProductID , p.ProductName having
avg_rating > 4;
- **GROUP BY and ORDER BY clauses to rank products by total sales.**
select ProductID , ProductName , sum(PricePerUnit*StockQuantity) as
Total_Sales from products
group by ProductID, ProductName order by Total_Sales ;

6. Identifying High-Value Customers

- **Calculate each customer's total spending.**
SELECT c.CustomerID, c.Name, SUM(od.Quantity * od.UnitPrice) AS
Total_Spending
FROM customers as c
JOIN orders as o ON c.CustomerID = o.CustomerID
JOIN order_details as od ON o.OrderID = od.OrderID
GROUP BY c.CustomerID, c.Name
ORDER BY Total_Spending ;
- **Rank customers based on their spending.**
SELECT c.CustomerID, c.Name, SUM(od.Quantity * od.UnitPrice) AS
Total_Spending,
RANK() OVER (ORDER BY SUM(od.Quantity * od.UnitPrice) DESC) AS
Rank_Position
FROM customers AS c
JOIN orders AS o ON c.CustomerID = o.CustomerID
JOIN order_details AS od ON o.OrderID = od.OrderID
GROUP BY c.CustomerID, c.Name
ORDER BY Total_Spending DESC;

- **Identify customers who have spent more than ₹5,000.**

```
select c.customerid, c.name, sum(od.quantity * od.unitprice) as total_spending
from customers as c
join orders as o on c.customerid = o.customerid
join order_details as od on o.orderid = od.orderid
group by c.customerid, c.name
having sum(od.quantity * od.unitprice) > 5000
order by total_spending desc;
```

7.Complex Aggregations and Joins

- **Join the Orders and OrderDetails tables to calculate total revenue per order.**

```
select
    o.orderid,
    sum(od.Quantity * od.UnitPrice) as total_revenue
from orders as o
join order_details as od on o.orderid = od.orderid
group by o.orderid
order by total_revenue desc;
```

- **Identify customers who placed the most orders in a specific time period.**

```
select c.customerid, c.name, count(o.orderid) as total_orders
from customers as c
join orders as o on c.customerid = o.customerid
where o.orderdate between '2025-01-01' and '2025-06-30'
group by c.customerid, c.name
order by total_orders desc;
```

- **Find the supplier with the most products in stock.**

```
select
    s.SupplierID,
    s.SupplierName,
    sum(p.StockQuantity) as total_stock
from suppliers as s
join products as p on s.SupplierID = p.SupplierID
group by s.SupplierID, s.SupplierName
order by total_stock desc
limit 1;
```

8.Normalization

- **Separate product categories and subcategories into a new table.**

```

Create table categories (
    CategoryID INT AUTO_INCREMENT PRIMARY KEY,
    CategoryName VARCHAR(100),
    SubCategoryName VARCHAR(100)
);
INSERT INTO categories (CategoryName, SubCategoryName)
SELECT DISTINCT Category, SubCategory
FROM products;

ALTER TABLE products
ADD CategoryID INT;

UPDATE products p
JOIN categories c
ON p.Category = c.CategoryName
AND p.SubCategory = c.SubCategoryName
SET p.CategoryID = c.CategoryID;

```

9.Subqueries and Nested Queries

➤ Identify the top 3 products based on sales revenue

```

select *
from (select p.ProductID,p.ProductName,sum(od.Quantity * od.UnitPrice) as
total_revenue
    from products as p
    join order_details as od on p.ProductID=od.ProductID
    group by p.ProductID, p.ProductName
) as product_revenue
order by total_revenue desc
limit 3;

```

➤ Find customers who haven't placed any orders yet.

```

select customerid,name
from customers
where customerid not in (
    select customerid from orders
);

```

	customerid	name
▶	0439fc2f-e186-46a8-8b2f-7c77d4b6d323	Daniel Nguyen
	024ebc32-77cc-4fca-b151-937425b5ab18	Erica Huber
	34688326-95cf-4da8-aa9d-bf28d48f7604	Gloria Dominguez
	5461729a-9e27-49e8-87c4-68097932e0dc	Hannah Meyer
	44189b67-cf70-4b7c-8315-499f872a588e	Jeremy Duran

Results:

- ✓ Design an efficient relational database.
- ✓ Generate SQL queries to answer key business questions.
- ✓ Create actionable reports for stakeholders.
- ✓ Apply ACID principles to ensure data integrity.
- ✓ Normalize datasets for optimized database structure.

Technical Tags:

SQL, DDL, DML, TCL, ACID Transactions, Database Normalization, E-commerce Analytics, Business Insights

Skills take away From

- ✓ This Project
- ✓ Data Modeling and ER Diagrams
- ✓ SQL Basics: Queries, Clauses, and Aggregations
- ✓ Database Management: DDL, DML, and Constraints
- ✓ Advanced SQL: Joins, Subqueries, and Normalization
- ✓ ACID Transactions and TCL Concepts
- ✓ Business Use Case Analysis

Conclusion:

The Amazon Fresh Analytics project successfully demonstrated the use of SQL for database management and business analysis. The project helped in organizing customer, product, supplier, and order data efficiently using relational database concepts. Various SQL operations such as joins, constraints, aggregations, and normalization were implemented to generate meaningful insights. Overall, the project improved practical knowledge in MySQL, data analysis, and real-world database management systems.